

Rapport - TP Cloud Computing

Deploiement automatise d'une application Flask sur Azure avec Terraform

Table des matieres

- 1. [Introduction](#)
- 2. [Architecture du projet](#)
- 3. [Etape 1 : Configuration de l'environnement](#)
- 4. [Etape 2 : Code Terraform \(Infrastructure as Code\)](#)
- 5. [Etape 3 : Backend Flask \(API CRUD\)](#)
- 6. [Etape 4 : Deploiement de l'infrastructure](#)
- 7. [Etape 5 : Tests et validation](#)
- 8. [Etape 6 : Suppression de l'infrastructure](#)
- 9. [Problemes rencontres et solutions](#)
- 10. [Conclusion](#)

1. Introduction

L'objectif de ce TP est de deployer automatiquement une application web sur Azure en utilisant Terraform comme outil d'Infrastructure as Code (IaC). Le projet comprend :

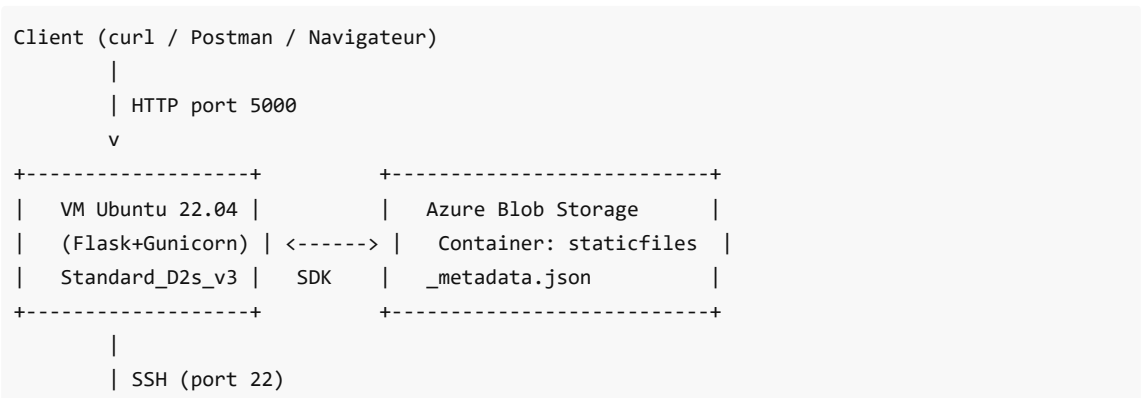
- Une VM Ubuntu hebergeant un backend Flask avec Gunicorn
- Un Azure Blob Storage pour le stockage de fichiers
- L'infrastructure reseau necessaire (VNet, Subnet, NSG, IP publique)
- Un script de provisioning automatique

L'application expose une API REST permettant d'effectuer des operations CRUD (Create, Read, Update, Delete) sur des fichiers stockes dans Azure Blob Storage.

Note sur les captures d'ecran

Toutes les captures d'ecran mentionnees dans ce rapport sont fournies dans un dossier separe dedie aux preuves visuelles (dossier `screenshots/`).

2. Architecture du projet



+-----+
Reseau Azure
VNet 10.0.0.0/16
Subnet /24
NSG (22, 5000)
IP publique
+-----+

Ressources Azure deployees

Ressource	Nom	Description
Resource Group	rg-tp-cloud	Groupe de ressources principal
Virtual Network	vnet-tp-cloud	Reseau virtuel (10.0.0.0/16)
Subnet	subnet-tp-cloud	Sous-reseau (10.0.1.0/24)
Public IP	pip-tp-cloud	IP publique statique de la VM
NSG	nsg-tp-cloud	Regles de securite (SSH + HTTP 5000)
NIC	nic-tp-cloud	Interface reseau de la VM
VM	vm-tp-cloud	Machine virtuelle Ubuntu 22.04 LTS
Storage Account	stcloudpierre2026	Compte de stockage blob
Container	staticfiles	Conteneur pour les fichiers uploades

3. Etape 1 : Configuration de l'environnement

Outils installes

- **Terraform** v1.14.7 : Outil d'IaC pour provisionner l'infrastructure Azure
- **Azure CLI** v2.84.0 : Interface en ligne de commande pour interagir avec Azure
- **PowerShell** : Terminal utilise sur Windows

Connexion Azure

```
az login
az account show --query "{name:name,id:id}" -o table
```

La subscription utilisee est **Azure for Students**.

```
PS C:\Users\Pierre\Documents\3eme ann  e\Cloud Computing> az account show --query "{nam
• e:name,id:id}" -o table
Name
-----
Azure for Students
PS C:\Users\Pierre\Documents\3eme ann  e\Cloud Computing> |
```

Initialisation Terraform

```
terraform init
```

Cette commande telecharge les providers necessaires :

- `hashicorp/azurerm v3.117.1` (gestion des ressources Azure)
- `hashicorp/tls v4.2.1` (generation de la cle SSH)

```
Azure for Students
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing> terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/tls from the dependency lock file
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Using previously-installed hashicorp/tls v4.2.1
- Using previously-installed hashicorp/azurerm v3.117.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing>
```

4. Etape 2 : Code Terraform (Infrastructure as Code)

Le code Terraform est reparti en 4 fichiers :

provider.tf

Configure le provider Azure (azurerm ~> 3.0) et passe le `subscription_id` en variable pour eviter de le coder en dur.

variables.tf

Declare 7 variables parametrables :

- `subscription_id` : identifiant de la subscription Azure
- `resource_group_name` : nom du resource group (default : `rg-tp-cloud`)
- `location` : region Azure (default : `France Central`)
- `vm_size` : taille de la VM (default : `Standard_D2s_v3`)
- `admin_username` : utilisateur SSH (default : `azureuser`)
- `storage_account_name` : nom globalement unique du storage (default : `stcloudpierre2026`)
- `container_name` : nom du conteneur blob (default : `staticfiles`)

Les valeurs sont fournies via `terraform.tfvars` (non versionne, un `.example` est fourni).

main.tf

Contient toutes les ressources Azure :

1. **Resource Group** : conteneur logique pour toutes les ressources
2. **VNet + Subnet** : reseau virtuel prive avec un sous-reseau /24
3. **IP publique** : allocation statique, SKU Standard
4. **NSG** : deux regles entrantes :
 - Port 22 (SSH) pour l'administration
 - Port 5000 (HTTP) pour l'application Flask
5. **NIC** : interface reseau associee au subnet et a l'IP publique
6. **Cle SSH** : generee automatiquement par Terraform (RSA 4096 bits) via le provider `tls` , ce qui evite de gerer manuellement les cle
7. **VM Linux** : Ubuntu 22.04 LTS, provisionne automatiquement via :
 - `provisioner "file"` : copie le dossier `app/` et le script `setup.sh` sur la VM
 - `provisioner "remote-exec"` : execute `setup.sh` avec les credentials du storage
8. **Storage Account** : compte de stockage blob avec replication LRS et regles CORS
9. **Container** : conteneur blob prive pour les fichiers

outputs.tf

Expose 8 sorties apres le deploiement :

- IP publique de la VM
- Nom d'utilisateur admin
- Cle private SSH (sensible)
- Nom et cle d'accès du storage (sensible)
- Endpoint blob et nom du conteneur
- URL de l'application (`http://<IP>:5000`)

5. Etape 3 : Backend Flask (API CRUD)

app/app.py

L'application Flask expose les endpoints suivants :

Methode	Route	Description
GET	/	Page d'accueil avec liste des endpoints
GET	/health	Healthcheck (verifie la connexion au storage)
POST	/files	Upload d'un fichier (multipart/form-data)
GET	/files	Liste tous les fichiers uploades
GET	/files/<id>	Telecharge un fichier par son ID
DELETE	/files/<id>	Supprime un fichier par son ID

Fonctionnement

- Chaque fichier uploadé reçoit un UUID unique

- Le fichier est stocké dans Azure Blob Storage sous le nom `{uuid}_{original_name}`
- Les métadonnées (id, nom original, type MIME, date) sont stockées dans un blob `_metadata.json`
- Le téléchargement et la suppression se font via l'ID du fichier

Dependances (requirements.txt)

- `flask>=3.0` : framework web
- `azure-storage-blob>=12.0` : SDK Azure pour le Blob Storage
- `python-dotenv>=1.0` : chargement des variables d'environnement
- `gunicorn>=21.0` : serveur WSGI pour la production

setup.sh (script de provisioning)

Le script effectue les opérations suivantes sur la VM :

1. Attend la libération des verrous apt (problème fréquent sur les VMs fraîchement créées)
2. Installe Python 3, pip et venv
3. Crée un environnement virtuel Python
4. Installe les dépendances depuis `requirements.txt`
5. Génère le fichier `.env` avec les credentials du storage (passés en arguments par Terraform)
6. Crée un service systemd `flask-app` pour lancer Gunicorn au démarrage
7. Active et démarre le service

6. Etape 4 : Déploiement de l'infrastructure

Commandes exécutées

```
terraform plan    # Vérification du plan d'exécution
terraform apply   # Déploiement effectif (confirmation requise)
```

`terraform plan` affiche les 10 ressources à créer et permet de vérifier avant d'appliquer.

```
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing> terraform plan
```

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

azurerm_linux_virtual_machine.vm will be created

```
+ resource "azurerm_linux_virtual_machine" "vm" {
  + admin_username           = "azureuser"
  + allow_extension_operations = true
  + bypass_platform_safety_checks_on_user_schedule_enabled = false
  + computer_name            = (known after apply)
  + disable_password_authentication = true
  + disk_controller_type     = (known after apply)
  + extensions_time_budget   = "PT1H30M"
  + id                       = (known after apply)
  + location                 = "francecentral"
  + max_bid_price            = -1
  + name                    = "vm-tp-cloud"
  + network_interface_ids    = (known after apply)
  + patch_assessment_mode    = "ImageDefault"
  + patch_mode               = "ImageDefault"
  + platform_fault_domain   = -1
  + priority                 = "Regular"
  + private_ip_address       = (known after apply)
  + private_ip_addresses     = (known after apply)
  + provision_vm_agent       = true
  + public_ip_address        = (known after apply)
  + public_ip_addresses     = (known after apply)
  + resource_group_name      = "rg-tp-cloud"
  + size                    = "Standard_D2s_v3"
  + virtual_machine_id       = (known after apply)
  + vm_agent_platform_updates_enabled = false

  + admin_ssh_key {
    + public_key = (known after apply)
    + username   = "azureuser"
  }

  + os_disk {
    + caching      = "ReadWrite"
    + disk_size_gb = (known after apply)
    + name         = (known after apply)
  }
}
```

BLACKBOX Agent [Open Website](#)

terraform apply cree les ressources dans l'ordre des dependances :

1. Resource Group
2. VNet, Subnet, IP publique, NSG (en parallele)
3. NIC et association NIC-NSG
4. Storage Account et Container
5. Cle SSH
6. VM (avec provisioning automatique)

```
# tls_private_key.ssh_key will be created
+ resource "tls_private_key" "ssh_key" {
  + algorithm          = "RSA"
  + ecdsa_curve        = "P224"
  + id                 = (known after apply)
  + private_key_openssh = (sensitive value)
  + private_key_pem     = (sensitive value)
  + private_key_pem_pkcs8 = (sensitive value)
  + public_key_fingerprint_md5 = (known after apply)
  + public_key_fingerprint_sha256 = (known after apply)
  + public_key_openssh = (known after apply)
  + public_key_pem     = (known after apply)
  + rsa_bits            = 4096
}
```

Plan: 11 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ app_url          = (known after apply)
+ ssh_private_key  = (sensitive value)
+ storage_account_name = "stcloudpierre2026"
+ storage_blob_endpoint = (known after apply)
+ storage_container_name = "staticfiles"
+ storage_primary_access_key = (sensitive value)
+ vm_admin_username  = "azureuser"
+ vm_public_ip      = (known after apply)
```

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

Resultat

Après le déploiement, Terraform affiche les sorties :

```
app_url = "http://<IP_PUBLIQUE>:5000"
vm_public_ip = "<IP_PUBLIQUE>"
```

```
azurerm_linux_virtual_machine.vm (remote-exec): (output suppressed due to sensitive value in config)
azurerm_linux_virtual_machine.vm (remote-exec): (output suppressed due to sensitive value in config)
azurerm_linux_virtual_machine.vm (remote-exec): (output suppressed due to sensitive value in config)
azurerm_linux_virtual_machine.vm: Creation complete after 2m18s [id=/subscriptions/439b28f0-3ed1-41e8-950a-eaca8072b134/resourceGroups/rg-tp-cloud/providers/Microsoft.Compute/virtualMachines/vm-tp-cloud]
```

Apply complete! Resources: 10 added, 0 changed, 0 destroyed.

Outputs:

```
app_url = "http://4.211.175.211:5000"
ssh_private_key = <sensitive>
storage_account_name = "stcloudpierre2026"
storage_blob_endpoint = "https://stcloudpierre2026.blob.core.windows.net/"
storage_container_name = "staticfiles"
storage_primary_access_key = <sensitive>
vm_admin_username = "azureuser"
vm_public_ip = "4.211.175.211"
```

7. Etape 5 : Tests et validation

Test 1 : Acces a l'application via l'IP publique

```
curl.exe http://<VM_IP>:5000/
```

Resultat : l'API repond avec la liste des endpoints disponibles.

```
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing> terraform output app_url
>> terraform output vm_public_ip
"http://4.211.175.211:5000"
"4.211.175.211"
```

Test 2 : Healthcheck (connexion au storage)

```
curl.exe http://<VM_IP>:5000/health
```

Resultat attendu :

```
{"status": "healthy", "storage": "connected"}
```

Ce test verifie que la VM communique correctement avec Azure Blob Storage.


```
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing> curl.exe http://4.211.175.211:5000/health
{"status":"healthy","storage":"connected"}
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing> curl.exe http://4.211.175.211:5000/
{"endpoints":{"DELETE /files/<file_id>":"Supprimer un fichier","GET /files":"Lister tous les fichiers","GET /files/<file_id>":"Télécharger un fichier","GET /health":"Vérifier la santé de l'application","POST /files":"Uploader un fichier (multipart/form-data, champ 'file')"},"message":"API Flask - Cloud Computing Project"}
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing>
```

Test 3 : Upload d'un fichier (CREATE)

```
echo "Hello Cloud Computing!" > test.txt
curl.exe -X POST -F "file=@test.txt" http://<VM IP>:5000/files
```

Resultat attendu :

```
{
  "message": "Fichier uploadé avec succès",
  "file": {
    "id": "<UUID>",
    "original_name": "test.txt",
    "blob_name": "<UUID>_test.txt",
    "content_type": "text/plain",
    "uploaded_at": "2026-03-12T..."
  }
}
```

```
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing> echo "Hello Cloud Computing!"
> test.txt
>> curl.exe -X POST -F "file=@test.txt" http://4.211.175.211:5000/files
{"file":{"blob_name":"d632ff38-a232-450a-9eb8-9a3e3b4d8ad6_test.txt","content_type":"text/plain","id":"d632ff38-a232-450a-9eb8-9a3e3b4d8ad6","original_name":"test.txt","uploaded_at":"2026-03-12T11:09:52.758819+00:00"},"message":"Fichier upload\u00e9 avec succ\u00e8s"}
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing>
```

Test 4 : Liste des fichiers (READ)

```
curl.exe http://<VM IP>:5000/files
```

Resultat : retourne la liste de tous les fichiers uploades avec leurs metadonnees.

```
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing> curl.exe http://4.211.175.211:5000/files
{"count":1,"files":[{"blob_name":"d632ff38-a232-450a-9eb8-9a3e3b4d8ad6_test.txt","content_type":"text/plain","id":"d632ff38-a232-450a-9eb8-9a3e3b4d8ad6","original_name":"test.txt","uploaded_at":"2026-03-12T11:09:52.758819+00:00"}]}
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing>
BLACKBOX Agent - Open Website
```

Test 5 : Telechargement d'un fichier (READ)

```
curl.exe http://<VM_IP>:5000/files/<file_id>
```

Resultat : retourne le contenu du fichier ("Hello Cloud Computing!").

```
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing> curl.exe http://4.211.175.211:5000/files/d632ff38-a232-450a-9eb8-9a3e3b4d8ad6
Warning: Binary output can mess up your terminal. Use "--output -" to tell curl to
Warning: output it to your terminal anyway, or consider "--output <FILE>" to save to
Warning: a file.
```

Test 6 : Suppression d'un fichier (DELETE)

```
curl.exe -X DELETE http://<VM_IP>:5000/files/<file_id>
```

Resultat attendu :

```
{"message": "Fichier supprimé avec succès"}
```

Verification : un GET /files apres suppression confirme que le fichier n'est plus dans la liste.

```
Warning: a file. curl.exe -X DELETE http://4.211.175.211:5000/files/d632ff38-a232-450a-9eb8-9a3e3b4d8ad6
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing>
{"id":"d632ff38-a232-450a-9eb8-9a3e3b4d8ad6","message":"Fichier supprimé avec succès"}
```

Test 7 : Verification dans Azure Blob Storage

On peut verifier directement dans le portail Azure que les fichiers sont bien stockes dans le conteneur `staticfiles` du storage account `stcloudpierre2026`.

Étiquettes (modifier) : Ajouter des étiquettes

Ressources Recommandations

Type est égal à tout
Emplacement est égal à tout
+ Ajouter un filtre

☐	Nom	Type	Emplacement
☐	nic-tp-cloud	Interface réseau	France Central
☐	nsg-tp-cloud	Groupe de sécurité réseau	France Central
☐	pip-tp-cloud	Adresse IP publique	France Central
☐	stcloudpierre2026	Compte de stockage	France Central
☐	vm-tp-cloud	Machine virtuelle	France Central
☐	vm-tp-cloud_OsDisk_1_3398f10f705459982639fb7a7276598	Disque	France Central
☐	vnet-tp-cloud	Réseau virtuel	France Central

8. Etape 6 : Suppression de l'infrastructure

Pour détruire toutes les ressources créées :

```
terraform destroy
```

Cette commande supprime dans l'ordre inverse toutes les ressources :

- VM et son disque
- NIC, NSG, IP publique
- Subnet, VNet
- Storage Account et Container
- Resource Group

```
Destroy complete! Resources: 11 destroyed.
PS C:\Users\Pierre\Documents\3eme année\Cloud Computing>
```

9. Problemes rencontres et solutions

Probleme 1 : Taille de VM non disponible

Symptome : Erreur `SkuNotAvailable` lors du `terraform apply` avec `Standard_B1s` et `Standard_B2s` dans la région France Central.

Cause : Certaines tailles de VM ne sont pas disponibles dans toutes les régions Azure, surtout avec les abonnements étudiants qui ont des quotas limités.

Solution : Changement de la taille de VM vers `Standard_D2s_v3` qui est disponible en France Central :

```
vm_size = "Standard_D2s_v3"
```

Probleme 2 : Region non autorisée

Symptome : Erreur `403 Forbidden` lors d'une tentative de déploiement sur `West Europe`.

Cause : La souscription Azure for Students applique des politiques (Azure Policy) qui restreignent les régions autorisées.

Solution : Utilisation de la region `France Central` (`francecentral`) qui est autorisee pour les subscriptions etudiantes.

Probleme 3 : Nom de Storage Account deja pris

Symptome : Erreur `StorageAccountAlreadyTaken` car le nom `stcloudprojectpierre` etait deja utilise globalement.

Cause : Les noms de Storage Account Azure doivent etre **globalement uniques** a travers tous les clients Azure.

Solution : Changement vers un nom unique : `stcloudpierre2026` .

Probleme 4 : Verrou apt lors du provisioning

Symptome : Le script `setup.sh` echoue (exit code 100) car `apt-get` est verrouille par un autre processus.

Cause : Sur les VMs fraichement creees, Ubuntu lance automatiquement des mises a jour en arriere-plan (`unattended-upgrades`), ce qui verrouille `apt` .

Solution : Ajout d'une boucle d'attente dans `setup.sh` qui patiente jusqu'a 10 minutes pour la liberation du verrou :

```
for i in $(seq 1 60); do
    if ! fuser /var/lib/dpkg/lock-frontent >/dev/null 2>&1; then
        break
    fi
    sleep 10
done
```

Probleme 5 : Provisioner file sous Windows

Symptome : Le `provisioner "file"` de Terraform copie `app/` comme un fichier unique au lieu d'un repertoire sur la VM.

Cause : Bug connu du provisioner file de Terraform sous Windows qui ne gere pas correctement la copie de repertoires.

Solution : Copie manuelle des fichiers via SCP, puis execution du script de provisioning via SSH :

```
scp -i ssh_key -r app/* azureuser@<IP>:/home/azureuser/app/
ssh -i ssh_key azureuser@<IP> "sudo bash /home/azureuser/setup.sh ..."
```

Probleme 6 : Desynchronisation du state Terraform

Symptome : Apres des echecs partiels de `terraform apply` , certaines ressources existent dans Azure mais pas dans le state Terraform, provoquant des conflits.

Cause : Lorsqu'un `apply` echoue a mi-parcours, les ressources deja creees ne sont pas enregistrees dans le state.

Solution : Utilisation de `terraform import` pour reimporter les ressources existantes dans le state :

```
terraform import azurerm_virtual_network.vnet /subscriptions/.../virtualNetworks/vnet-tp-
cloud
```

```
terraform import azurerm_storage_account.storage
/subscriptions/.../storageAccounts/stcloudpierre2026
```

10. Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux du Cloud Computing :

- **Infrastructure as Code** : Terraform permet de définir, versionner et reproduire l'infrastructure de manière déclarative. Toutes les ressources Azure sont décrites dans des fichiers `.tf`.
- **Provisioning automatisé** : Le script `setup.sh` installe et configure automatiquement l'application sur la VM, rendant le déploiement reproductible.
- **Stockage cloud** : Azure Blob Storage offre un stockage scalable et résilient pour les fichiers, accessible via le SDK Python.
- **API REST** : Le backend Flask expose une API CRUD complète pour la gestion de fichiers, démontrant l'interaction entre une application et les services cloud.
- **Sécurité** : Les fichiers sensibles (`terraform.tfvars` , `.env` , `.tfstate`) sont exclus du dépôt Git via `.gitignore` . Les clés SSH sont générées automatiquement par Terraform et marquées comme sensibles.

Stack technique

Composant	Technologie
IaC	Terraform (azurerm ~> 3.0)
Cloud	Microsoft Azure
Compute	VM Ubuntu 22.04 LTS
Backend	Flask + Gunicorn
Stockage	Azure Blob Storage
SDK	azure-storage-blob (Python)
Provisioning	Bash (setup.sh)
Versioning	Git + GitHub